# Informe de Laboratorio 03

## Tema: JavaScript (Backend)

| Nota |
|---|
| |

| Estudiantes | Escuela | Asignatura |
|---|---|---|
| Deyci Camargo<br>Viviana Linares<br>Suemy Figueroa<br>Ysabel Zambrano | Escuela Profesional de Ingeniería de Sistemas | Programación<br>Semestre: I<br>Código: 2502117 |

| Laboratorio | Tema | Duración |
|---|---|---|
| 03 | JavaScript (Backend) | 04 horas |

| Semestre académico | Fecha de inicio | Fecha de entrega |
|---|---|---|
| 2026 - A | Del 20 Abril 2026 | Al 27 Mayo 2026 |

## 1. Tarea

- Implementar una agenda con Node.js y Express.
- Las actividades deben guardarse como archivos de Markdown.
- Utilizar Docker para desplegar el sitio web: '/lab03'.
- Automatizar el despliegue de la tarea mediante un Dockerfile y aplicar todas las recomendaciones para crear la imagen y el contenedor.
- Utilizar Git para evidenciar su trabajo.
- Enviar trabajo al profesor en un repositorio GitHub Privado, dándole permisos como colaborador.

## 2. Equipos, materiales y temas utilizados

- HTML
- CSS
- JavaScript
- Visual Studio Code
- Docker
- Cuenta en GitHub con el correo institucional.

# 3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/YsabelZambrano/desarrollodeaplicacionesweb.git`
- URL para el laboratorio 03 en el Repositorio GitHub.
- `https://github.com/YsabelZambrano/desarrollodeaplicacionesweb/tree/main/lab03`

# 4. URL del Video y Acceso al sitio web

- URL del Video en Youtube
- `https://youtu.be/bmZmjsJE-Ek`
- URL para acceder al sitio web.
- `http://localhost:8095`

# 5. Paso a paso del desarrollo de la aplicación web

## 5.1. Creación de la estructura base (HTML)

- Se creó la estructura principal de la página web utilizando HTML. Se definieron los elementos básicos como encabezado, botones y contenedores para la información de los integrantes.

Listing 1: index.html

```html
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="css/style.css">
  <title>Agenda</title>
</head>
<body>

<h1 class="titulo">   Agenda</h1>

<div class="main">

  <div class="card">
    <h2>      Editar Evento</h2>

    <label>Fecha</label>
    <input type="date" id="fecha">

    <label>Hora</label>
    <input type="time" id="hora">

    <label>Descripcin</label>
    <textarea id="desc" placeholder="Descripcin"></textarea>

    <div class="actions">
      <button class="btn-primary" onclick="crear()">Guardar / Actualizar</button>
```

```
      <button class="btn-cancel" onclick="limpiar()">Cancelar</button>
    </div>
  </div>

  <div class="card">
    <h2>    Eventos Programados</h2>

    <div class="stats">
      <div class="stat">
        <span id="totalEventos">0</span>
        <p>Total Eventos</p>
      </div>
      <div class="stat">
        <span id="totalDias">0</span>
        <p> Das   nicos </p>
      </div>
    </div>

    <div id="lista"></div>

    <h2>    Vista mensual</h2>

    <div class="cal-header">
      <button onclick="cambiarMes(-1)">  </button>
      <span id="mesActual"></span>
      <button onclick="cambiarMes(1)">  </button>
    </div>

    <div id="calendario" class="calendario"></div>
  </div>

</div>

<script>
let editando = null;
let fechaActual = new Date();

function formatearFecha(f) {
  return f.replaceAll("-", ".");
}

function formatearHora(h) {
  return h.replace(":", ".");
}

async function cargar() {
  const res = await fetch('/api/eventos');
  const data = await res.json();

  const contenedor = document.getElementById('lista');
  contenedor.innerHTML = '';

  const grupos = {};

  data.forEach(e => {
    if (!grupos[e.fecha]) grupos[e.fecha] = [];
```

```
    grupos[e.fecha].push(e);
  });

  for (const f in grupos) {
    grupos[f].sort((a, b) => a.hora.localeCompare(b.hora));
  }

  document.getElementById('totalEventos').innerText = data.length;
  document.getElementById('totalDias').innerText = Object.keys(grupos).length;

  for (const fecha in grupos) {
    const bloque = document.createElement('div');
    bloque.className = 'dia';

    bloque.innerHTML = `<h3>${fecha}</h3>`;

    grupos[fecha].forEach(e => {
      const item = document.createElement('div');
      item.className = 'evento';

      item.innerHTML = `
        <div>
          <span class="hora">${e.hora}</span> - ${e.descripcion}
        </div>
        <div>
          <button class="btn-edit"
              onclick="editar('${e.fecha}','${e.hora}','${e.descripcion}')">Editar</button>
          <button class="btn-delete"
              onclick="eliminarYRecargar('${e.fecha}','${e.hora}')">Eliminar</button>
        </div>
      `;

      bloque.appendChild(item);
    });

    contenedor.appendChild(bloque);
  }

  generarCalendario(data);
}

async function crear() {
  const fechaInput = document.getElementById('fecha').value;
  const horaInput = document.getElementById('hora').value;
  const desc = document.getElementById('desc').value;

  if (!fechaInput || !horaInput || !desc) {
    alert("Completa todos los campos");
    return;
  }

  const fecha = formatearFecha(fechaInput);
  const hora = formatearHora(horaInput);

  if (editando) {
    await eliminar(editando.fecha, editando.hora);
```

```
    editando = null;
  }

  await fetch('/api/eventos', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({ fecha, hora, descripcion: desc })
  });

  limpiar();
  cargar();
}

async function eliminar(fecha, hora) {
  await fetch('/api/eventos', {
    method: 'DELETE',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({ fecha, hora })
  });
}

async function eliminarYRecargar(f, h) {
  await eliminar(f, h);
  cargar();
}

function editar(f, h, d) {
  document.getElementById('fecha').value = f.replaceAll(".", "-");
  document.getElementById('hora').value = h.replace(".", ":");
  document.getElementById('desc').value = d;

  editando = { fecha: f, hora: h };
}

function limpiar() {
  document.getElementById('fecha').value = '';
  document.getElementById('hora').value = '';
  document.getElementById('desc').value = '';
  editando = null;
}

function cambiarMes(dir) {
  fechaActual.setMonth(fechaActual.getMonth() + dir);
  cargar();
}

function generarCalendario(data) {
  const cont = document.getElementById('calendario');
  const titulo = document.getElementById('mesActual');

  cont.innerHTML = '';

  const  ao  = fechaActual.getFullYear();
  const mes = fechaActual.getMonth();

  const nombres = ["Enero","Febrero","Marzo","Abril","Mayo","Junio",
```

```
                  "Julio","Agosto","Septiembre","Octubre","Noviembre","Diciembre"];

  titulo.innerText = `${nombres[mes]} ${ao}`;

  const primerDia = new Date(ao, mes, 1).getDay();
  const diasMes = new Date(ao, mes + 1, 0).getDate();

  const fechasEventos = new Set(data.map(e => e.fecha));

  for (let i = 0; i < primerDia; i++) {
    cont.appendChild(document.createElement('div'));
  }

  for (let d = 1; d <= diasMes; d++) {
    const div = document.createElement('div');
    div.className = 'dia-cal';
    div.innerText = d;

    const fecha = `${ao}.${String(mes+1).padStart(2,'0')}.${String(d).padStart(2,'0')}`;

    if (fechasEventos.has(fecha)) {
      div.classList.add('tiene-evento');

      const punto = document.createElement('div');
      punto.className = 'punto';
      div.appendChild(punto);
    }

    cont.appendChild(div);
  }
}

cargar();
</script>

</body>
</html>
```

## 5.2. Creación del JavaScript (JS)

- Se creó la estructura de JS que funciona como una agenda simple (tipo calendario) donde se puede guardar, listar y eliminar eventos desde un servidor web.

Listing 2: index.js

```
const express = require('express');
const fs = require('fs');
const path = require('path');

const app = express();

app.use(express.json());


app.use(express.static(path.join(__dirname, 'pub')));
```

```
app.get('/api/eventos', (req, res) => {
  const base = './agenda';
  let eventos = [];

  if (!fs.existsSync(base)) return res.json([]);

  const fechas = fs.readdirSync(base);

  fechas.forEach(fecha => {
    const rutaFecha = path.join(base, fecha);

    if (!fs.lstatSync(rutaFecha).isDirectory()) return;

    const horas = fs.readdirSync(rutaFecha);

    horas.forEach(hora => {
      const rutaArchivo = path.join(rutaFecha, hora);

      if (!hora.endsWith('.md')) return;

      const contenido = fs.readFileSync(rutaArchivo, 'utf-8');

      eventos.push({
        fecha,
        hora: hora.replace('.md', ''),
        descripcion: contenido
      });
    });
  });

  res.json(eventos);
});


app.post('/api/eventos', (req, res) => {
  let { fecha, hora, descripcion } = req.body;

  if (!fecha || !hora || !descripcion) {
    return res.status(400).json({ error: 'Datos incompletos' });
  }

  const dir = `./agenda/${fecha}`;

  if (!fs.existsSync(dir)) {
    fs.mkdirSync(dir, { recursive: true });
  }

  fs.writeFileSync(`${dir}/${hora}.md`, descripcion);

  res.json({ ok: true });
});


app.delete('/api/eventos', (req, res) => {
```

```
  const { fecha, hora } = req.body;

  const dir = `./agenda/${fecha}`;

  if (!fs.existsSync(dir)) {
    return res.json({ ok: false });
  }

  const archivos = fs.readdirSync(dir);

  const archivo = archivos.find(a => a.startsWith(hora));

  if (archivo) {
    fs.unlinkSync(`${dir}/${archivo}`);
    console.log('Eliminado:', archivo);
  } else {
    console.log('No encontrado para borrar:', hora);
  }

  res.json({ ok: true });
});


app.listen(3000, () => {
  console.log('http://localhost:3000/');
});
```

## 5.3. Diseño visual con CSS

- Luego se aplicaron estilos con CSS para mejorar la presentación de la página, como colores, tipografía y organización de los elementos.

Listing 3: Diseño visual con CSS

```
    body {
        font-family: 'Segoe UI', sans-serif;
        background: #eef2f7;
        margin: 0;
      }

      .titulo {
        text-align: center;
        margin-top: 20px;
        color: #2b4c7e;
      }

      .main {
        display: flex;
        gap: 40px;
        padding: 50px;
        justify-content: center;
      }

      .card {
        background: white;
```

```
        padding: 25px;
        border-radius: 14px;
        width: 420px;
        box-shadow: 0 12px 30px rgba(0,0,0,0.08);
      }
```

### 5.4. Pruebas de funcionamiento

Se realizaron pruebas en el navegador para verificar que:

- La estructura HTML se visualiza correctamente.
- Los estilos CSS se aplican sin errores.
- El botón ejecuta correctamente el evento en JavaScript.

### 5.5. Estructura de laboratorio 01

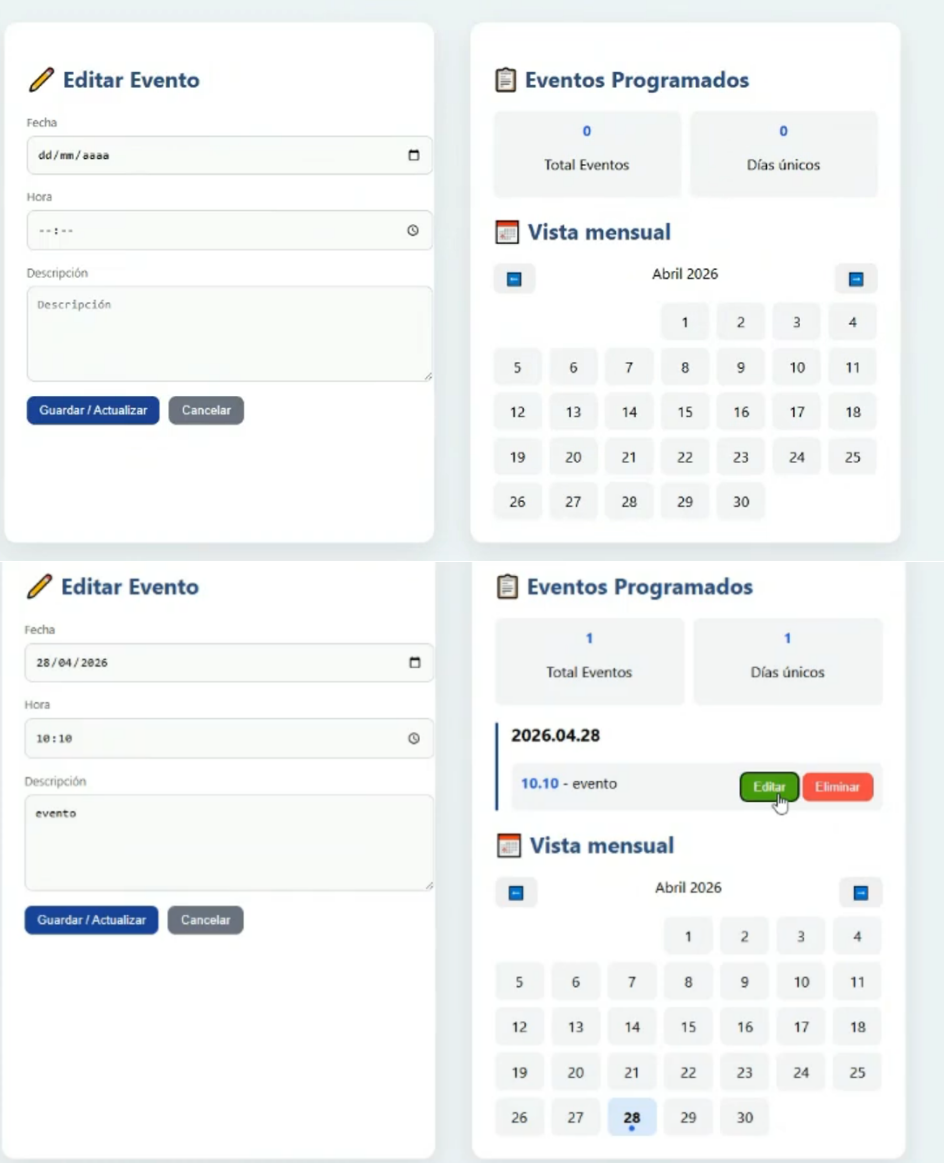
- El contenido que se entrega en este laboratorio es el siguiente:

```
lab03/
   |--- .dockerignore
   |--- .gitignore
   |--- Dockerfile
   |--- index.js
   |--- package-lock.json
   |--- package.json
   |--- README.md
+------pub
   |--- index.html
   +------css
   |--- style.css
```

## 6. Resultados

La aplicación web desarrollada permite visualizar la información de los integrantes del grupo de manera organizada y estilizada. Se logro implementar:

- Diseño responsivo básico.
- Interacción mediante botones.
- Separación de estuctura HTML, diseño CSS y lógica JavaScript.


Editar Evento
Fecha
dd/mm/aaaa
Hora
--:--
Descripción
Descripción
Guardar / Actualizar
Cancelar
Eventos Programados
0
Total Eventos
0
Días únicos
Vista mensual
Abril 2026
Editar Evento
Fecha
28/04/2026
Hora
10:10
Descripción
evento
Guardar / Actualizar
Cancelar
Eventos Programados
1
Total Eventos
1
Días únicos
2026.04.28
10.10 - evento
Editar
Eliminar
Vista mensual
Abril 2026

# 7. Conclusiones

- Se logró desarrollar una aplicación web básica utilizando HTML, CSS y JavaScript.
- Se reforzaron habilidades en la creación de interfaces y manejo de JavaScript.
- Este laboratorio permitió comprender la importancia de separar la estructura, el diseño y la funcionalidad en el desarrollo web.

# 8. Rúbricas

## 8.1. Auto - calificación

| Informe | |
|---|---|
| Deyci Andrea Camargo Ramos | 100 % |
| Viviana Flobele Linares Taco | 100 % |
| Suemy Alexandra Figueroa Laruta | 100 % |
| Ysabel Alejandra Zambrano Cruz | 100 % |

## 8.2. Rúbrica para el contenido del Informe y demostración

| **Ítem** | **Descripción** | **Puntaje** |
|---|---|---|
| Informe | El informe está completo, utiliza la plantilla y tiene un acabado impecable. (Debe estar en el repositorio Github y Classroom) | 2 |
| Video | El video es preciso y muestra la ejecución del contenedor en la terminal y la navegación por los ejercicios. (Video en Youtube. URL en Informe, Classroom y README.md) | 2 |
| Crea evento | El ejercicio está completo y contiene todas las recomendaciones. | 4 |
| Lista eventos | El ejercicio está completo y contiene todas las recomendaciones. | 3 |
| Edita evento | El ejercicio está completo y contiene todas las recomendaciones. | 3 |
| Elimina evento | El ejercicio está completo y contiene todas las recomendaciones. | 3 |
| README.md | El laboratorio cuenta con un README.md necesario para desplegar la aplicación web. | 2 |
| Prueba | Se toman en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo. | 0 |
| | **Total** | **19** |

# 9. Referencias


- https://www.w3schools.com/java/default.asp
- https://www.geeksforgeeks.org/insertion-sort/